SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**

Question Count: 5

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

NAME:K.Saipriya
REGNO: 192421296

COURSECODE: CSA1510

COURSENAME: CloudComputing

1. HDFS-Based Storage Solution for a Media Platform

Problem Analysis

The media platform stores large amounts of videos, images, audio files, and metadata. The design must satisfy three major constraints:

1. Data availability must remain above 99.9%.
2. Replication factor must not exceed 3.
3. Storage growth must remain cost efficient.

Proposed Design

A suitable solution is an HDFS-based distributed storage architecture with a replication factor of 3 for critical data.

Architecture:

Media Upload → Ingestion Layer → HDFS → Data Processing → Metadata/Analytics

The HDFS cluster can contain:

- **NameNode:** manages file-system metadata.
- **Secondary/Standby NameNode:** supports metadata recovery/high availability.
- **DataNodes:** store actual media blocks.
- **HDFS Replication:** maintains three copies of important blocks.
- **YARN:** manages processing resources.
- **Hive:** provides SQL-based analysis of media metadata.

Storage Strategy

The media files are divided into large HDFS blocks and distributed across multiple DataNodes. A **replication factor of 3** is used for important media data.

Instead of keeping every copy on the same physical machine, HDFS should place replicas across different nodes and preferably different racks.

For example:

BlockA→DataNode1
ReplicaA→DataNode2
Replica A → DataNode 3

If one DataNode fails, another replica can immediately serve the data.

Cost-Efficient Growth

To control storage costs:

- Use commodity servers instead of expensive high-end storage.
- Increase DataNodes when storage requirements grow.
- Use different storage tiers for frequently and rarely accessed media.
- Compress metadata and text-based information.
- Store older, rarely accessed content in a lower-cost storage tier.
- Avoid unnecessary replication beyond the maximum factor of 3.

Availability

A **99.9%+ availability requirement** can be supported through:

- HDFS replication factor of 3.
- Multiple DataNodes.
- Rack-aware block placement.
- NameNode high availability using Active and Standby NameNodes.
- Automatic failure detection and recovery.
- Regular health monitoring.

Justification

This design satisfies the constraints because replication factor 3 provides fault tolerance without exceeding the allowed limit. Rack-aware placement prevents a single rack failure from making all copies unavailable. Adding DataNodes horizontally allows the platform to grow without replacing the entire storage system.

Therefore, the architecture provides high availability, fault tolerance, and cost-efficient scalability while remaining within the replication constraint.

2. MapReduce Workflow Within a 20-Minute Batch Window

Problem Analysis

The system needs to process large log files within a strict **20-minute batch window**, while the number of cluster nodes is limited.

Therefore, the main constraints are:

1. Processing must finish within **20 minutes**.
2. Cluster resources are limited.
3. Tasks must be distributed efficiently.
4. Data movement between nodes should be minimized.

Proposed MapReduce Design

The workflow can be designed as:

Log Files → HDFS → Input Splitting → Mapper → Combiner → Reducer → Output

Suppose web logs contain:

Timestamp | UserID | URL | Status | ResponseTime

The objective could be to calculate:

- Number of requests.
- Number of errors.
- Requests per URL.
- Average response time.

Mapper

The Mapper reads each log record and generates key-value pairs.

Example:

URL → 1

For response-time analysis:

URL → ResponseTime

Since HDFS distributes the log files across DataNodes, multiple Mapper tasks can process different input blocks simultaneously.

Combiner

A **Combiner** should be used whenever possible.

For example, instead of sending:

/home→1
/home→1
/home → 1

to the Reducer individually, the Combiner can produce:

/home → 3

This reduces network traffic and improves execution time.

Reducer

Reducers aggregate the intermediate values.

Example:

/home → [3, 5, 2, 7]

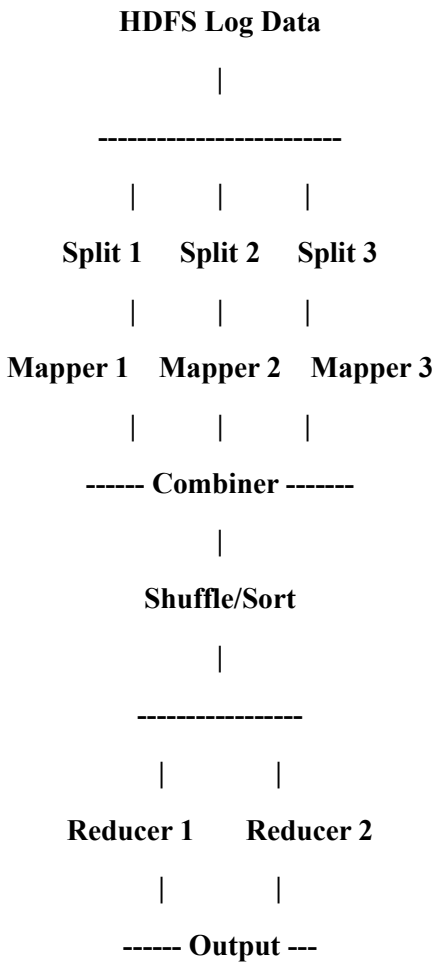
Output:

/home → 17 requests

Task Distribution

Because the number of nodes is limited, the cluster should use controlled parallelism.

Example:



Meeting the 20-Minute Constraint

The 20-minute limit can be achieved by:

- Increasing Mapper parallelism within available nodes.
- Keeping input files appropriately sized.
- Using a Combiner to reduce shuffle data.
- Using efficient serialization.
- Avoiding unnecessary MapReduce stages.
- Partitioning data appropriately among Reducers.
- Keeping computation close to the data.
- Monitoring slow-running tasks and avoiding reducer bottlenecks.

Justification

MapReduce is suitable because HDFS automatically distributes the input data, allowing multiple Mapper tasks to work simultaneously. The Combiner reduces network traffic, while Reducers perform parallel aggregation.

If a particular Reducer receives excessive data, the partitioning strategy can be adjusted to prevent a data-skew bottleneck.

Thus, the proposed workflow makes the best use of limited nodes while targeting completion within the 20-minute batch window.

3. YARN Scheduler for ETL, Reporting and Machine Learning

Problem Analysis

The YARN cluster runs three different workloads:

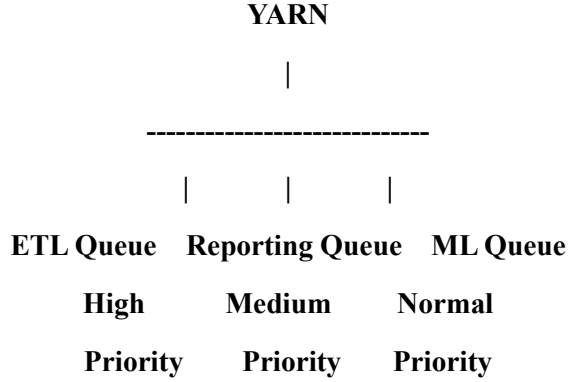
- **ETL jobs**
- **Reporting jobs**
- **Machine Learning jobs**

The scheduler must satisfy:

1. **Fairness** between workloads.
2. **Higher priority for ETL.**
3. **Recovery when a node fails.**

Proposed Scheduler Design

A suitable approach is to configure **YARN Capacity Scheduler** with separate queues.



Example allocation:

Queue	Priority	Resource Allocation
ETL	High	50%
Reporting	Medium	20%
Machine Learning	Normal	30%

These values can be adjusted according to the organization's workload.

ETL Queue

ETL receives the highest priority because data ingestion and transformation may be required before reporting and ML processes can operate.

The scheduler can provide minimum guaranteed resources to the ETL queue while allowing unused resources to be temporarily borrowed by other queues.

Reporting Queue

Reporting jobs receive a medium priority.

This prevents reporting from consuming the entire cluster while still ensuring that business reports are generated regularly.

Machine Learning Queue

ML jobs can use the remaining resources and can receive additional resources when the ETL and reporting queues are idle.

Fairness

Fairness can be maintained by:

- Allocating minimum resources to each queue.
- Preventing one workload from monopolizing the cluster.
- Allowing unused resources to be shared.
- Using queue limits and user limits.

Node Failure Recovery

YARN provides fault handling through the ResourceManager and NodeManagers.

If a NodeManager fails:

Node Failure → ResourceManager Detects Failure → Failed Tasks Identified → Tasks Rescheduled → Another Node Executes Task

For important services, ResourceManager High Availability can also be configured using an Active/Standby ResourceManager arrangement.

Justification

The scheduler prioritizes ETL without completely starving reporting and ML workloads. Queue-based resource allocation ensures fairness, while YARN's monitoring and task rescheduling provide recovery from node failures.

Therefore, the design provides priority, fairness, resource utilization, and fault recovery.

4. Ingesting Data from SAN, NAS and Operational Databases

Problem Analysis

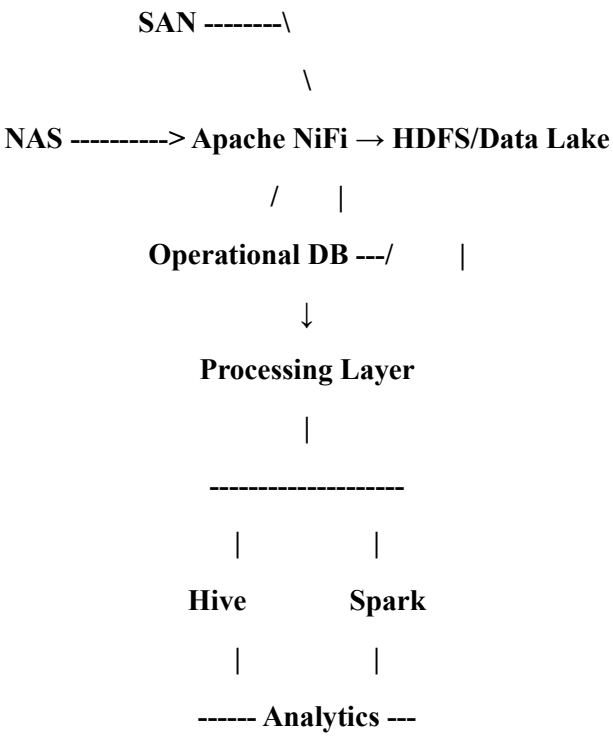
The enterprise receives data from three major sources:

- **SAN:** Storage Area Network.
- **NAS:** Network Attached Storage.
- **Operational databases:** MySQL, PostgreSQL, Oracle, etc.

The architecture must provide:

1. Reliable backup.
2. Minimal duplication.
3. Integration with the big data platform.
4. Reliable ingestion.

Proposed Architecture



SAN and NAS Ingestion

Apache NiFi can be used to collect files from SAN and NAS.

NiFi processors can:

- Detect new files.
- Validate files.
- Add metadata.

- Route files.
- Transfer files to HDFS.
- Track data lineage.

Database Ingestion

For operational databases, an ETL/ingestion process can extract only new or changed records.

For example:

Database → Incremental Extraction → NiFi → HDFS

Instead of repeatedly copying the entire database, the system can use:

- Timestamp-based incremental extraction.
- Primary-key based extraction.
- Change Data Capture where supported.

This significantly reduces duplication.

Backup Reliability

The HDFS platform provides multiple copies of important data through replication.

For example:

OriginalBlock→DataNode
Replica→DataNode2
Replica → DataNode 3

Critical metadata and configuration should also be backed up separately.

Duplicate Reduction

NiFi can assign unique identifiers and maintain metadata such as:

- File name.
- Source.
- Timestamp.
- File size.
- Checksum.

A checksum can be used to identify identical files before storing another copy.

Data Organization

HDFS can use separate directories such as:

/data/raw/san/

/data/raw/nas/

/data/raw/database/

/data/archive/

This gives clear separation between raw, processed, and archived data.

Justification

NiFi provides controlled ingestion and data provenance, while HDFS provides scalable distributed storage. Incremental database extraction and checksum-based file validation reduce unnecessary duplication.

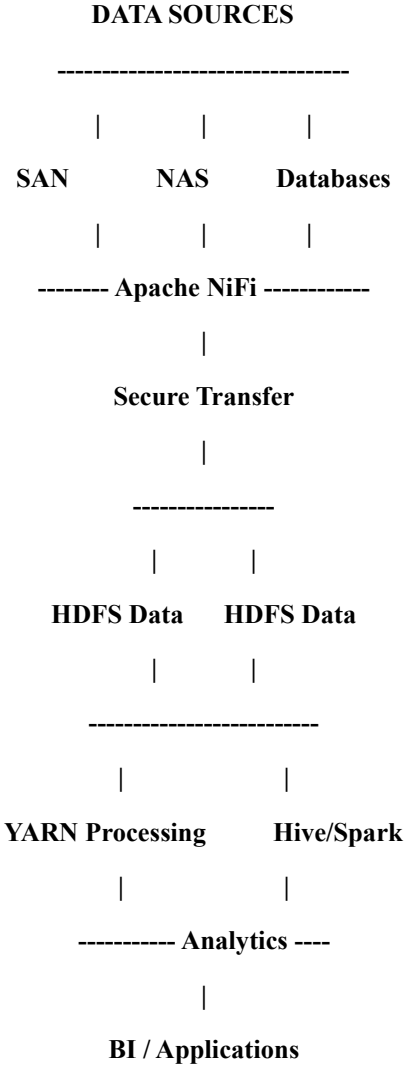
5. End-to-End Hadoop Ecosystem with Secure Data Movement and High Availability

Problem Analysis

The enterprise requires a complete Hadoop ecosystem with:

1. Secure data movement.
2. High availability.
3. Integration with Apache NiFi or ETL pipelines.
4. Scalable processing and storage.

Proposed Architecture :



1.Secure Data Movement

Apache NiFi can be placed at the ingestion layer.

Security mechanisms include:

- HTTPS/TLS for data transfer.
- Authentication between systems.
- Encryption during transmission.
- Access control.
- Secure credentials.
- Data provenance tracking.

Sensitive information should not be transmitted using unsecured protocols.

2. HDFS High Availability

The HDFS layer can use:

- Active NameNode.
- Standby NameNode.
- Multiple DataNodes.
- HDFS replication factor of 3.
- Rack-aware block placement.

Architecture:

HDFS HA

|

| |

Active NameNode Standby NameNode

| |

------- Metadata --

|

| | |

DataNode 1 DataNode 2 DataNode 3

If the Active NameNode fails, the Standby NameNode can take over.

3. YARN High Availability

YARN can also use an Active/Standby Resource Manager architecture.

YARN

|

| |

Active ResourceMgr Standby ResourceMgr

|

NodeManagers

If a worker node fails, YARN can detect the failure and reschedule affected tasks on healthy nodes.

4. Apache NiFi Integration

NiFi provides the connection between external sources and Hadoop.

A typical pipeline is:

Source → NiFi → Validation → Transformation → HDFS → YARN/Spark → Hive → BI

NiFi can also perform:

- Routing.
- Filtering.
- Data transformation.
- Error handling.
- Monitoring.
- Data provenance.

5. Processing Layer

YARN manages cluster resources, while processing frameworks such as MapReduce or Spark execute workloads.

Hive can provide SQL-based access to large datasets.

The overall architecture therefore uses the major Hadoop ecosystem components appropriately.

Constraint Satisfaction

Constraint	Design Solution
Secure data movement	NiFi + TLS/HTTPS + authentication
HDFS high availability	Active/Standby NameNodes
Storage fault tolerance	HDFS replication
Processing availability	YARN resource management and task rescheduling
Data ingestion	Apache NiFi
ETL integration	NiFi/ETL pipeline
Scalability	Add DataNodes/worker nodes
Monitoring	NiFi and Hadoop monitoring
Data analysis	Hive/Spark/MapReduce

Justification

This architecture separates data ingestion, storage, processing, and analytics, making the system easier to manage and scale. NiFi securely connects external data sources to HDFS, while HDFS replication and NameNode high availability protect against infrastructure failures. YARN manages processing resources and reschedules workloads when nodes fail.

Therefore, the proposed system satisfies the requirements for secure data movement, high availability, scalable storage, reliable processing, and Hadoop ecosystem integration.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured